ENCE260: Embedded Systems

Lecture 2: Data Input/Output (I/O)

How does a micro interact with the rest of the system?

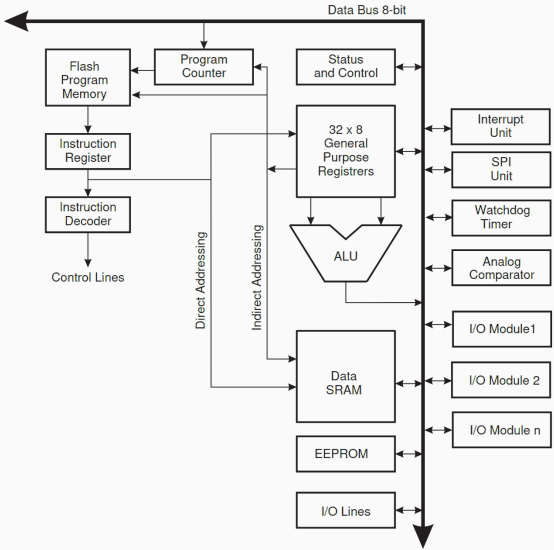
Revision

Why is a microprocessor inconvenient for embedded system use?

1. They have too much memory.
2. They don't have enough memory. ✓
3. They have too much processing power.
4. They use too much power. ✓

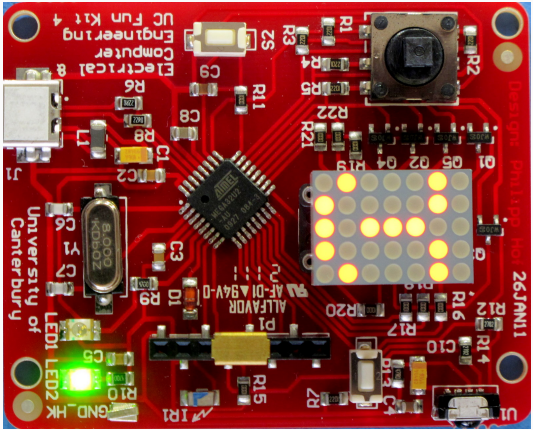
AVR Architecture Overview

- Taken directly from the datasheet, which you can find on Learn.
- Has a Harvard architecture (split memory for data and instructions).
- Uses memory-mapped I/O, which is what we'll focus on today.



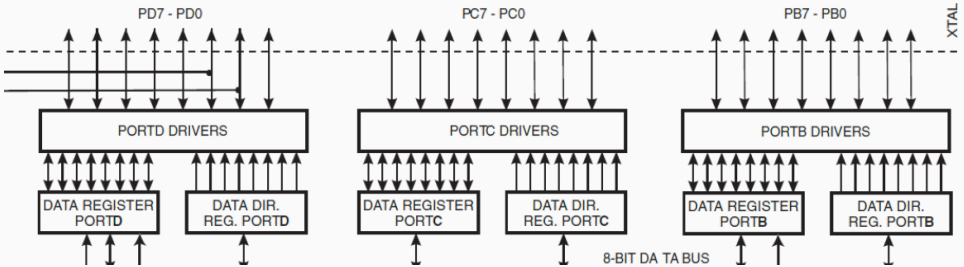
Programmable I/O

All microcontrollers have programmable I/O (PIO) pins to interface with external hardware. The typical examples are buttons and Light Emitting Diodes (LEDs).



Data I/O on the ATmega32U2

- The Atmega32U2 has 23 PIO pins, split over three 8-bit ports: Port B, Port C, and Port D. PC3 is unused.
- Each pin can be configured as an input (e.g. the button on `PD7`) or an output (e.g. the LED on `PC2`).



I/O Port Registers

- The I/O Port registers control the flow of input/output to/from the microcontroller.
- Each I/O Port has three 8-bit registers which control its operation.
- For a generic Port x (where x = B, C, or D), there registers are:
  - `DDRx` : The **D**ata **D**irection **R**egister for Port **x**.
  - `PORTx` : The output data register for Port **x**.
  - `PINx` : The **P**ort **IN**put register for Port **x**.
- Across the three registers, a specific bit controls a specific pin. For example, pin `PC2` is controlled by the bit 2 in each of `DDRC`, `PORTC`, and `PINC`.

Data Direction Register `DDRx`

- For each pin in the port, controls the direction that data flows.
  - If data is coming in from e.g. a button, the pin is an input.
  - If data is being sent out to e.g. an LED, the pin is an output.
- A `DDRx` bit value of zero means the pin is an input.
- A bit value of one means the pin is an output.
- Data Direction Registers *usually* only need to be configured once, at startup.

0 = input

1 = output

Output Data Register `PORTx`

- For each pin in the port, controls the output state.
- This lets us set the voltage on each pin.
- A `PORTx` bit value of zero means the pin is set low.
- A bit value of one means the pin is set high.

$$0 = \text{low} = 0\,\text{V}$$

$$1 = \text{high} = 5\,\text{V}$$

Port Input Register `PINx`

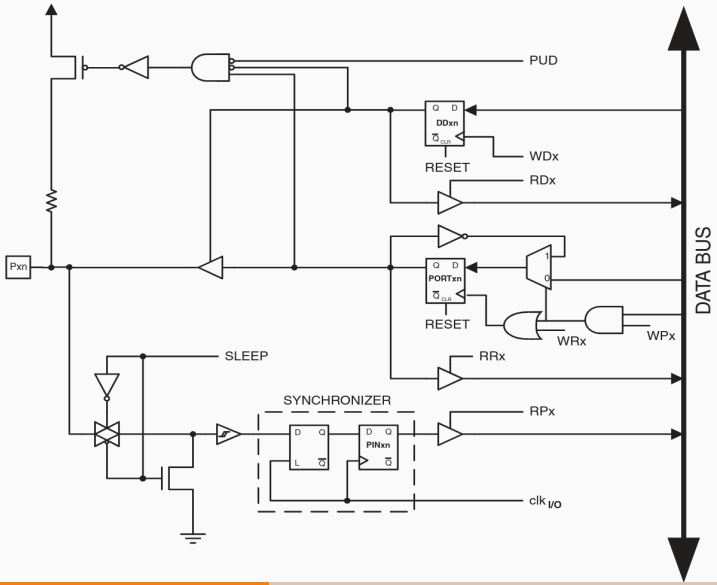
- For each pin in the port, stores the input state.
- The micro itself doesn't control the pin state for an input.
- This register reports the voltage that's on the pin.
- A `PINx` bit value of zero means the pin is held low.
- A bit value of one means the pin is held high.

Same as with output:

$$0 = \text{low} = 0\,\text{V}$$

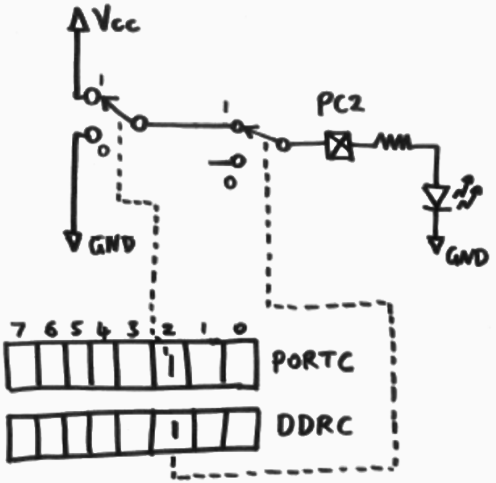
$$1 = \text{high} = 5\,\text{V}$$

Example Configuration



Example Configuration

- Say we want to turn on e.g. the LED connected to `PC2`.
- Need to set `DDRC` bit 2 = 1. This makes `PC2` an output.
- Need to set `PORTC` bit 2 = 1. This makes `PC2` high (5 V).



Bitwise Operators

In C, we use the following bitwise (i.e., manipulating bit-by-bit) operators:

`~`	Bitwise complement (NOT): flips all the bits
`&`	Bitwise AND
`|`	Bitwise OR
`^`	Bitwise exclusive or (XOR)
`<<`	Left shift: moves all bits left an amount
`>>`	Right shift: moves all bits right an amount

Bit Manipulation

- We use the bitwise operators to manipulate specific bits in registers.
- We typically move a 1 or 0 into a specific bit in an I/O register.
- E.g., the C expression `1 << 2`:

```
1           = 0000 0001
1 << 2      = 0000 0100
~(1 << 2)   = 1111 1011
```

Bitmasks

- Specifically constructed binary values (such as 0000 0100 and 1111 1011) are called *bitmasks*.
- We can use bitwise operators to use these to modify specific bits, while leaving all others untouched.

  E.g.

$$\begin{array}{rcc} & xxxx & xxxx \\ | & 0000 & 0100 \\ \hline & xxxx & x1xx \end{array}$$

Bitmasks can modify multiple bits at a time, e.g. with 0001 0100.

Bit Operations

The operations we can do are:

- Setting a bit (ensures that it is 1).

```
PORTC = PORTC | (1<<2);
```

```
PORTC |= (1<<2);
```

- Clearing a bit (ensures that it is 0).

```
PORTC &= ~(1<<2);
```

- Toggling a bit (changes bit value; 0 →1, 1 →0).

```
PORTC ^= (1<<2);
```

- Testing a bit (return 1 if bit is set, else 0).

```
if ((PINC & (1<<2)) != 0) {
```

BIT() Macro

- We can also define a macro to make life easier:

```
#define BIT(X) (1<<(X))
```

- This macro has a single argument `X`, which the preprocessor substitutes.

  E.g. `BIT(2)` → `(1<<(2))`

- We can set `PC2` and `PC4` by constructing a bitmask:

```
PORTC |= ( BIT(4) | BIT(2) );
```

  Equivalent to `PORTC |= 0b00010100;`

DDR Config Example

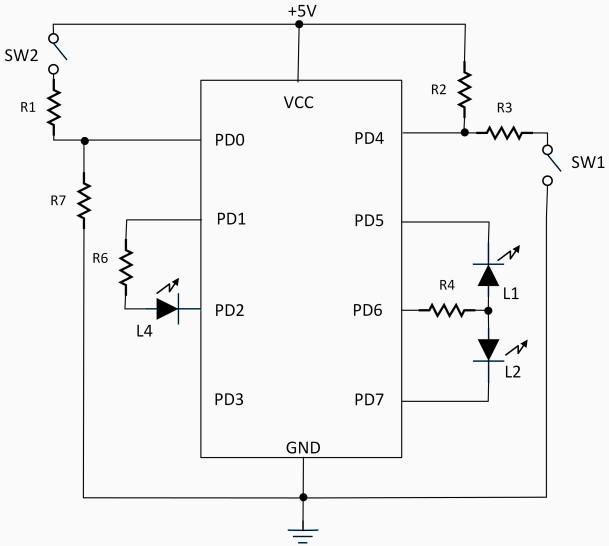
What value should be stored in `DDRD`?

0 = input (switch)

1 = output (LED)

`DDRD = PD7 ... PD0`

`DDRD = 1110 x110`



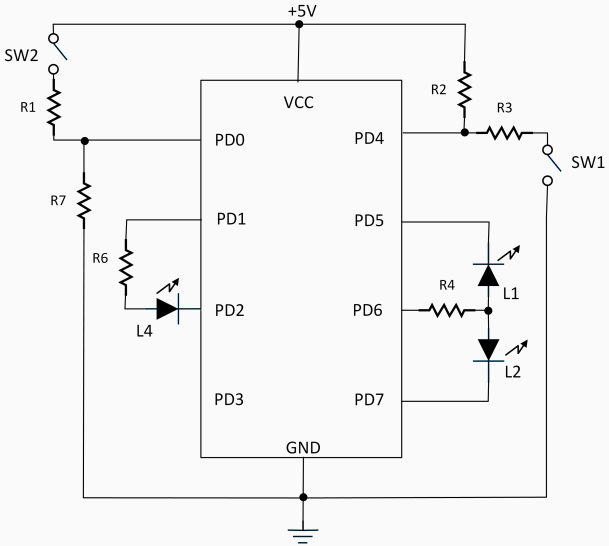
DDR Config Example

How would we configure a newly-connected LED to on PD3?

1 = output (LED)

```
DDRD = DDRD | (1<<3);
```

$$\begin{array}{rr} 1110 & x110 \\ \mid \quad 0000 & 1000 \\ \hline 1110 & 1110 \end{array}$$



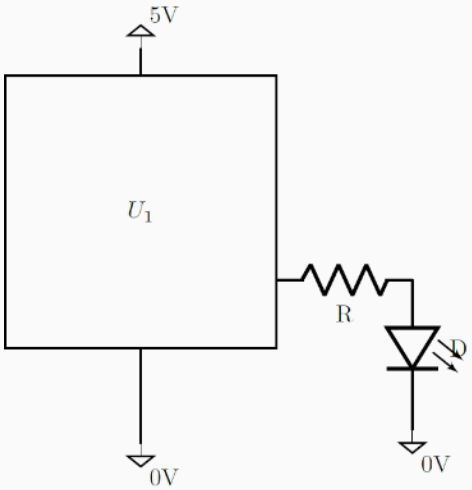
LED Details

- LEDs don't linearly resist their flowing current.
- This means we need to limit the current ourselves, both to control the brightness and to prevent the LED from blowing.
- Such a resistor is called a "current-limiting resistor", and should always be present when using LEDs.
- LEDs burn a roughly constant amount of voltage across them (e.g. $\approx 1.7\,\mathrm{V}$ for red LEDs), so the resistor must disapate the extra voltage.



Output Example

To drive the user LED (PC2) on the UCFK4 we need to:



- Configure PC2 as an output.

```
DDRC |= (1<<2);
```

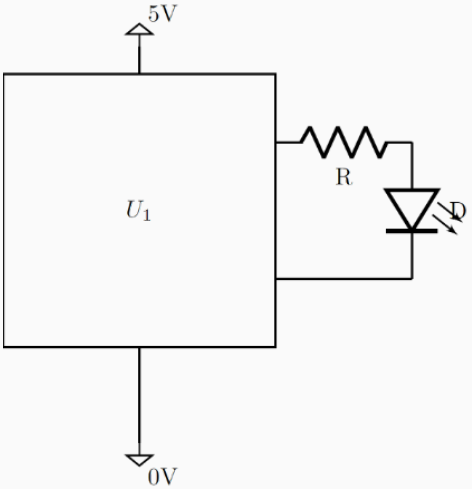
- Set PC2 high.

```
PORTC |= (1<<2);
```

- Leave the other bits in any modified registers the same.

LED Between Two Pins

To, for examle, turn on an LED connected between PB2 (supply) and PB5 (ground):



- Configure PB2 and PB5 as outputs.

```
DDRB |= (1<<2) | (1<<5);
```

- Set PB2 high.

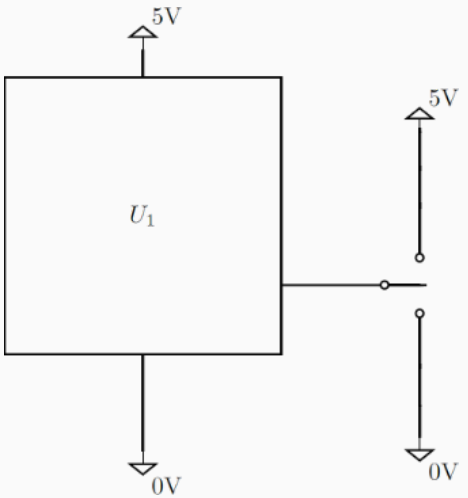
```
PORTB |= (1<<2);
```

- Set PB5 low.

```
PORTB &= ~(1<<5);
```

- Leave the other bits in any modified registers the same.

Input Example



To check the voltage on the pin `PD7` we need to:

- Configure `PD7` as an input.

```
DDRD &= ~(1<<7);
```

- Test state of `PD7`.

```
if ((PIND & (1<<7)) != 0) {
```

- Leave the other bits in any modified registers the same.

Key Concepts

- I/O pins are grouped into ports, which are each controlled by the registers `DDRx`, `PORTx`, and `PINx`.
- We configure these registers using bitwise operations and bitmasks.
- Different hardware configurations are driven/read in different ways. Always look at the circuit first!

Homework

Work through all the operations on the “Bit Operations” slide step-by-step.

Use dummy values and work through the operations by hand with examples, and convince yourself that they work.

This should also hopefully help you understand intuitively *why* they work.